Implementación de Métodos RESTful, Seguridad JWT, Validaciones y Frontend Angular

> **Proyecto:** Sistema de Gestión de Residuos de Aparatos Eléctricos y Electrónicos (RAEE) y Ventas
> **Capa Backend:** Spring Boot 3.3.4, Spring Security 6, JJWT 0.12.6, Jakarta Validation, Spring Data JPA
> **Capa Frontend:** Angular 18 (Standalone Components), Bootstrap 5, SASS, RxJS

---

1. Métodos RESTful en Controladores (`show`, `create`, `update`, `delete`)

Los controladores implementan la convención RESTful estandarizada para todos los recursos principales:

```
flowchart TD
    Client["Cliente HTTP (Angular / Postman)"]
    subgraph Spring Boot Controller Layer
        C1["GET /api/recurso<br/>(index / listar)"]
        C2["GET /api/recurso/{id}<br/>(show / buscarPorId)"]
        C3["POST /api/recurso<br/>(create / guardar)"]
        C4["PUT /api/recurso/{id}<br/>(update / actualizar)"]
        C5["DELETE /api/recurso/{id}<br/>(delete / eliminar)"]
    end
    Client --> C1 & C2 & C3 & C4 & C5
```

Tabla de Endpoints y Códigos HTTP

- **`index` (`GET /api/{recurso}`):** Retorna la colección completa de entidades (`200 OK`).
- **`show` (`GET /api/{recurso}/{id}`):** Retorna la entidad solicitada por ID (`200 OK`) o dispara `ResourceNotFoundException` (`404 Not Found`).
- **`create` (`POST /api/{recurso}`):** Valida la carga útil con `@Valid` y persiste la entidad (`201 Created`).
- **`update` (`PUT /api/{recurso}/{id}`):** Valida y actualiza los campos (`200 OK`) o dispara `ResourceNotFoundException` (`404 Not Found`).
- **`delete` (`DELETE /api/{recurso}/{id}`):** Elimina el registro (`204 No Content`) o retorna `404 Not Found`.

Implementado en:

- ProductoController.java
- ClienteController.java
- VentaController.java
- CategoriaController.java

---

2. Incorporación de Validaciones (Jakarta Bean Validation)

Se han añadido restricciones declarativas para garantizar que no ingresen registros nulos, vacíos o fuera de rango:

2.1. Reglas en Entidades y DTOs

- **En Producto.java:**
  - `@NotBlank(message = "El nombre del producto es obligatorio")`: Evita cadenas vacías o espacios en blanco.
  - `@NotNull(message = "El precio es obligatorio")`: Evita valores nulos.
  - `@PositiveOrZero(message = "El precio no puede ser negativo")`: Precios ≥ 0.0.
  - `@Min(value = 0, message = "El stock no puede ser negativo")`: Cantidad en inventario ≥ 0.
- **En Cliente.java:**
  - `@NotBlank(message = "El nombre del cliente es obligatorio")`.
  - `@Size(min = 8, max = 11, message = "La identificación debe contener entre 8 y 11 caracteres")`: Valida longitud para DNI (8) o RUC (11).
  - `@Email(message = "El formato de correo electrónico no es válido")`.
  - `@Pattern(regexp = "^[0-9+() -]*$", message = "El teléfono sólo puede contener números o guiones")`.
- **En Venta.java:**
  - `@NotNull(message = "El cliente es obligatorio para registrar la venta")`.
  - `@NotEmpty(message = "La venta debe incluir al menos un detalle de producto")`.

---

3. Respuestas HTTP Adecuadas y Manejo Global de Excepciones

Se centralizó el tratamiento de errores mediante `@RestControllerAdvice`, generando una estructura JSON uniforme:

```
{
  "timestamp": "2026-11-03T01:19:35",
  "status": 400,
  "error": "Error de Validación",
  "message": "Uno o más campos contienen valores incorrectos o incompletos",
  "path": "/api/productos",
  "validationErrors": {
    "nombre": "El nombre del producto es obligatorio",
    "precio": "El precio no puede ser negativo"
  }
}
```

Excepciones Gestionadas:

1. `ResourceNotFoundException` $\rightarrow$ **`404 Not Found`**
2. `MethodArgumentNotValidException` $\rightarrow$ **`400 Bad Request`** (con desglose campo por campo en `validationErrors`).
3. `BadRequestException` $\rightarrow$ **`400 Bad Request`**.
4. `BadCredentialsException` $\rightarrow$ **`401 Unauthorized`**.
5. `AccessDeniedException` $\rightarrow$ **`403 Forbidden`**.
6. `Exception` $\rightarrow$ **`500 Internal Server Error`**.

---

4. Configuración de Spring Security y Autenticación JWT

Se diseñó la arquitectura de seguridad basada en Tokens JWT sin estado (*stateless*):

```
sequenceDiagram
    autonumber
    actor User as Usuario (Angular)
    participant AuthCtrl as AuthController (/api/auth/login)
    participant Sec as Spring Security
    participant JWT as JwtUtil
    participant API as Endpoints Protegidos (/api/productos)

    User->>AuthCtrl: POST credenciales (username, password)
    AuthCtrl->>Sec: AuthenticationManager.authenticate()
    Sec-->>AuthCtrl: Autenticación OK
    AuthCtrl->>JWT: generarToken(username, rol)
    JWT-->>AuthCtrl: Retorna Token JWT (Bearer)
    AuthCtrl-->>User: 200 OK { token, username, rol }

    Note over User, API: Siguientes Peticiones con Cabecera HTTP
    User->>API: GET /api/productos (Authorization: Bearer <token>)
    API->>JWT: JwtAuthenticationFilter.validarToken()
    JWT-->>API: Token Válido + Rol Extraído
    API-->>User: 200 OK [Datos de Inventario]
```

Componentes Clave:

1. **JwtUtil.java:** Emite y valida tokens firmados con HMAC-SHA256, expiración configurable (24 horas) y *claims* que incluyen el rol del usuario.
2. **JwtAuthenticationFilter.java:** Interceptor que extrae la cabecera `Authorization: Bearer <token>`, valida la firma y carga el contexto de seguridad en `SecurityContextHolder`.
3. **SecurityConfig.java:**
   - Define `SessionCreationPolicy.STATELESS`.
   - Habilita `@EnableMethodSecurity(prePostEnabled = true)`.
   - Utiliza `BCryptPasswordEncoder` para encriptar contraseñas.

---

5. Definición y Control de Roles (`ROLE_ADMIN` y `ROLE_VENDEDOR`)

El sistema implementa control de acceso basado en roles (RBAC):

Rol	Alcance de Permisos	Anotación Spring Security
`ROLE_ADMIN`	Acceso total: Crear, consultar, editar y **eliminar** cualquier recurso.	`@PreAuthorize("hasRole('ADMIN')")`
`ROLE_VENDEDOR`	Acceso operativo: Crear, consultar y modificar productos, clientes y registrar ventas. *(No puede eliminar registros)*.	`@PreAuthorize("hasAnyRole('ADMIN', 'VENDEDOR')")`

Usuarios Precargados Automáticamente (DataInitializer.java):

- **Administrador:** `admin` / `admin123` (Rol: `ROLE_ADMIN`)
- **Vendedor:** `vendedor` / `vendedor123` (Rol: `ROLE_VENDEDOR`)

---

6. Configuración de CORS para Angular

En SecurityConfig.java se habilitó la comunicación cruzada:

- **Orígenes Autorizados:** `http://localhost:4200` y `http://127.0.0.1:4200`.
- **Métodos Permitidos:** `GET, POST, PUT, DELETE, OPTIONS, PATCH`.
- **Cabeceras Permitidas:** `Authorization, Content-Type, Accept, X-Requested-With`.
- **Exposición de Cabecera:** `Authorization`.
- **Credenciales:** `allowCredentials: true`.

---

7. Estructura del Proyecto Frontend en Angular

El proyecto se encuentra ubicado en el directorio `frontend/`:

```
frontend/src/app/
├── core/
│   ├── guards/auth.guard.ts         # Bloquea rutas privadas si no hay JWT
│   ├── interceptors/jwt.interceptor.ts # Inyecta 'Authorization: Bearer <token>'
│   └── services/auth.service.ts     # Manejo de inicio/cierre de sesión
├── models/                          # Interfaces TypeScript (Producto, Cliente, Venta, etc.)
├── services/                        # Servicios de conexión HTTP a la API REST
└── components/
    ├── navbar/                      # Barra de navegación con indicador de rol
    ├── login/                       # Formulario de autenticación con validaciones
    ├── inicio/                      # Dashboard con KPIs y métricas del campus
    ├── productos/                   # Gestión completa de inventario RAEE
    ├── clientes/                    # Directorio de clientes con filtros
    └── ventas/                      # Módulo de ventas con carrito reactivo y totales
```

---

8. Componentes Generados y Funcionalidades

1. **NavbarComponent:** Muestra el usuario activo, su rol en un badge de color y botón de cierre de sesión.
2. **LoginComponent:** Permite ingresar credenciales, ver u ocultar contraseña, e informa errores de autenticación.
3. **InicioComponent:** Dashboard con 4 tarjetas KPI (Total Equipos, Clientes, Diagnósticos pendientes y Recaudación) y accesos directos.
4. **ProductosComponent:** Tabla con buscador en tiempo real, filtro por estado (`RECIBIDO` , `OPERATIVO` , `EN_MANTENIMIENTO`), modal de registro/edición y restricción de eliminación sólo para Administrador.
5. **ClientesComponent:** Catálogo de clientes con validaciones para DNI/RUC y email.
6. **VentasComponent:** Sistema de caja registradora con selección de cliente, adición de productos con cantidad, cálculo dinámico de subtotales y emisión de la transacción.

---

9. Aplicación de Interpolación, Property Binding y Directivas

En cada componente se aplicaron las técnicas esenciales de Angular:

- **Interpolación (`{{ }}`):**

```
<span class="text-primary">{{ p.precio | currency:'PEN':'S/. ' }}</span>
<h3>{{ totalClientes }}</h3>
```

- **Property Binding (`[propiedad]`):**

```
<button class="btn btn-success" [disabled]="prodForm.invalid">Guardar</button>
<tr [class.table-warning]="p.stock === 0">
<span class="badge" [ngClass]="p.estado === 'OPERATIVO' ? 'badge-operativo' : 'badge-recibido'">
```

- **Event Binding (`(evento)`):**

```
<button class="btn btn-outline-danger" (click)="eliminar(p.id!)">Eliminar</button>
<form (ngSubmit)="onLogin(loginForm)">
```

- **Two-way Data Binding (`[(ngModel)]`):**

```
<input type="text" [(ngModel)]="productoForm.nombre" name="nombre" required />
```

- **Directivas Estructurales (`*ngIf` , `*ngFor`):**

```
<!-- Renderizado condicional por rol -->
<button *ngIf="authService.isAdmin()" class="btn btn-danger">Eliminar</button>

<!-- Iteración de listas -->
<tr *ngFor="let item of carrito; let i = index">
```

---

10. Diseño con Bootstrap 5, SASS, Flexbox y Grid

- **CSS Grid (styles.scss):** Utilizado en la clase `.dashboard-grid` para adaptar automáticamente el número de columnas según la resolución de pantalla:

```
.dashboard-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1.5rem;
}
```

- **Flexbox:** Distribución de tarjetas KPI (`d-flex align-items-center justify-content-between`) y alineación en barras de navegación y encabezados.
- **Componentes Bootstrap 5:** Clases de formularios (`form-control` , `input-group` , `form-select`), modales con `modal-backdrop` , alertas interactivas (`alert alert-success alert-dismissible`) y botones de estado (`badge`).

---

11. Batería de Pruebas de Seguridad y Validación

Se implementó el test automatizado SecurityAndValidationTest.java con **MockMvc**:

```
[INFO] Running com.example.demo.SecurityAndValidationTest
- 1. Autenticación fallida con credenciales incorrectas (401) -> PASS
- 2. Autenticación exitosa y generación de Token JWT (200) -> PASS
- 3. Acceso denegado a ruta protegida sin token JWT (403) -> PASS
- 4. Validación de campos: Rechazo de producto con nombre vacío (400) -> PASS
- 5. Control de Acceso por Roles: VENDEDOR no puede eliminar productos (403) -> PASS

[INFO] Results:
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```